

Evaluation of a distributed deep learning framework as a reference architecture for a cloud environment

Attila Farkas^{1,3}, Krisztián Póra¹, Sándor Szénási², Gábor Kertész^{2,1}, and Róbert Lovas¹

¹Institute for Computer Science and Control (SZTAKI), Eötvös Lóránd Research Network (ELKH)

²John von Neumann Faculty of Informatics, Óbuda University

³Doctoral School of Applied Informatics and Applied Mathematics, Óbuda University

Abstract—Distributed deep learning is becoming increasingly important due to the size of deep neural networks. The sheer volume of the input datasets used in the process can have a significant negative effect on the training time of the model. As a consequence of physical limitations, the resources of a single node cannot be scaled to a sufficient level for efficient training. As such, different parallel and distributed solutions exist in order to solve this problem. In this paper, we measured the scalability of Horovod, a distributed deep learning framework (available as a reference architecture) with different parameters on the research infrastructure called ELKH Cloud. The experiment was conducted in order to verify the scalability of the framework on the general purpose infrastructure, which was not designed primarily for distributed deep learning applications. Measurements assessing model accuracy were also performed, for the purpose of validating the distributed training process.

I. INTRODUCTION

Nowadays, deep learning is a rapidly growing area and is taking effects on our daily life [1]. The increased number of available training data and the complexity of deep neural network models amplify the problem of the training process resource demand. Due to the physical limitations of a single node, modern deep learning frameworks provide different distributed solutions and strategies to allocate the resources of multiple nodes for a single training process. This computational model is called distributed deep learning (DDL). Each framework provides this functionality with different strategies, architecture and communication protocols, but there are some general frameworks that incorporate multiple different deep learning frameworks as well. The other crucial part is the resource requirements of a DDL framework for efficient, distributed execution. Cloud computing or High-Performance Computing (HPC) [2] environments can act as a solution for this resource demand, but in most cases, these infrastructures are built for general purpose usage, and are not dedicated for DDL processes. Therefore, in this paper, we evaluated a DDL framework with different datasets and parameters on a cloud based research infrastructure.

II. DEEP LEARNING

A neural network is a computational model that consists of many neurons built into different layers as computational units. Networks with one or a few hidden layers are referred to as shallow neural networks. These networks are capable of solving basic classification and regression problems. Networks

with more hidden layers are known as deep neural networks. Deep neural networks with more hidden layers contain more neurons, therefore they have more trainable parameters as well. The size of the training dataset is also increased to provide a more accurate neural network model. These two properties have negative effects on the training time. To increase the accuracy and decrease the training time of the neural network model, activation functions and optimization methods can be used. Because of these features, the deep neural networks can be used for more complicated tasks such as speech recognition, machine translation or image processing.

A. Limitation of Deep Learning

Because of the size of deep neural networks and the size of the training dataset, the necessary computational power is also increased for the training process of these networks. CPU based training is ideal only for shallow neural networks. The robust parallel execution of modern GPU cards can speed-up the training process of deep neural networks, however these cards have their own limitations as well. The slow communication between the CPU and the GPU card slows down the transfer of the training data. The memory size of GPU cards limits the size of the model and the applied batch size for the training. Due to physical constraints, scalability within a single node is also limited. Therefore, different parallel and distributed solutions can be utilized in order to speed up the training process, which is the previously introduced methodology of distributed deep learning (DDL).

B. Distributed Deep Learning

The DDL methodology enables the utilization of multiple nodes for the same training process. The two main distributed training strategies are data-parallel and model-parallel [3] execution. With the data-parallel approach, the nodes involved in the training process different segments of the same input dataset. In model-parallel mode, the corresponding nodes are training different segments of the same model. This approach is a more communication-intensive method, therefore it is mainly used on a single node with multiple execution units. The applied communication protocol is another significant aspect of distributed training. The various modern deep learning frameworks utilize different protocols. Tensorflow [4] provides

the Distributed package for DDL. This library supports different strategies for the distributed training, uses gRPC for communication and can utilize CPU, GPU and TPU resources. PyTorch [5] provides a Distributed package as well, which supports different communication protocols like Gloo [6], MPI [7], NVIDIA NCCL [8] and also supports CPU and GPU based execution. There are general DDL frameworks like Horovod [9], which can provide the same DDL strategies for multiple different deep learning frameworks. [10]

C. Horovod

Horovod is an open-source DDL framework originally developed by Uber. It provides the functionality to distribute the training process of a deep neural network model between different nodes. It supports TensorFlow, Keras [11], PyTorch, Apache MXNet [12] and Spark [13] frameworks. Horovod provides an easy-to-use framework for distributed deep learning, which enables the distribution of training with just a few minor changes to the original code. It supports data-parallel execution, which means that each node of the cluster will process a different part of the same input dataset during training. For the distributed training strategy, Horovod improves Baidu's original idea [14] and introduces the Ring-allreduce strategy. With this method, the nodes are organized into a ring, and they communicate with their neighbors throughout the training. In this structure, each of the N nodes will exchange data with two of its peers $2 \cdot (N-1)$ times. Horovod uses MPI for inner-node communication, and can also utilize NVIDIA NCCL 2.0 during training.

III. MEASUREMENTS

Based on our experience, it is a common approach by researchers to use general purpose infrastructures, such as computational cloud environments for machine learning based research, due to the availability of these infrastructures, especially in Hungary. Therefore, the goal of our measurements was to examine how well the Horovod framework can be utilized on a computational cloud environment which is not specifically built for DDL applications. We planned to examine the scalability of the framework with different parameters and datasets for the machine learning model training.

A. Environment

For the measurements we used the federated, community cloud of the Eotvos Lorand Research Network, called ELKH Cloud. The main goal of ELKH Cloud is supporting the Hungarian scientist community with virtualized computing resources. We deployed the Horovod framework as a reference architecture (RA) on the ELKH Cloud with different configurations for the measurements.

1) *ELKH Cloud*: The ELKH Cloud consists of two different sites, the site of the Institute of Computer Science and Control (abbreviated as SZTAKI) and the site of the Wigner Research Centre for Physics (abbreviated as WDC). This science cloud provides a combined 5904 vCPUs, with 28TB RAM, 1248TB HDD and 338TB SSD storage and a total of

68 GPUs with 2400GB RAM at this time. The measurements were executed on the SZTAKI site which contains 10 compute nodes, each with 2x Intel Xeon Gold 6230R 26-Core CPUs, 768GB of RAM and 4x NVIDIA Tesla V100 [15] GPU 32GB with SXM2 socket and NVLink connection. The GPU cards are provided in the form of virtual GPUs (vGPU) to the virtual machines based on the NVIDIA vCompute Server solution. Additionally, there are 6 HDD and 6 SSD storage nodes, each with 192TB and 92TB of raw storage, respectively. The virtual machines have 10Gbps networking among each other. The ELKH Cloud is connected to the Hungarian academic backbone with a 100Gbps network. The cloud is based on OpenStack [16] Wallaby, and the storage backend is built using Ceph [17] Pacific.

2) *Horovod reference architecture*: We developed and utilized the Horovod reference architecture [18], which was created for the ELKH Cloud project. The reference architecture methodology leads to a Platform-as-a-Service (PaaS) solution, following the Infrastructure-as-Code (IaC) principle. As such, the RA is made up of different descriptor files and contains every necessary component to build up a Horovod cluster on a cloud provider. Furthermore, the RA fulfills different non-functional requirements, such as scalability, high-availability, configurability and security aspects. In order to ensure these principles, we also created a continuous integration and continuous delivery (CI/CD) workflow based on GitLab CI [19] to periodically test the RA against these requirements. We used Terraform [20] as a cloud orchestrator and Ansible [21] as a configuration management tool to create the Horovod RA. The architecture of the Horovod RA is presented on the Figure 1.

The Horovod framework is deployed as Docker containers on the virtual machines of the cluster to provide a more portable solution. For this, we extended the original Horovod Docker image with a JupyterLab IDE [22] to enable users to handle the system and an NFS [23] client component for data-sharing inside the cluster. While all nodes use the same base image, some components built into the image, such as JupyterLab can be activated using environment variables. This is used on the appointed master node, which also contains the NFS server component. The virtual machines are created by Terraform, and the configuration of the whole cluster is handled by Ansible.

3) *Configuration*: For the measurements, we used the SZTAKI site of the ELKH Cloud. We built up a cluster of 8 virtual machines using the g2.2xlarge flavor. Each machine contained 16 vCPU cores, 64GB RAM, 1 NVIDIA Tesla V100 GPU with 32GB RAM as a vGPU, 32GB SSD storage for the root disc and a dedicated 10Gbps network interface. We used a 1TB SSD based volume for the NFS based file-sharing.

B. Parameters

Thanks to the configurable and automatic deployment features of the Horovod RA we can easily deploy the whole environment with different configurations. For the measurements, we used an official TensorFlow benchmark script with a ResNet-101 [24] convolutional neural network. To measure

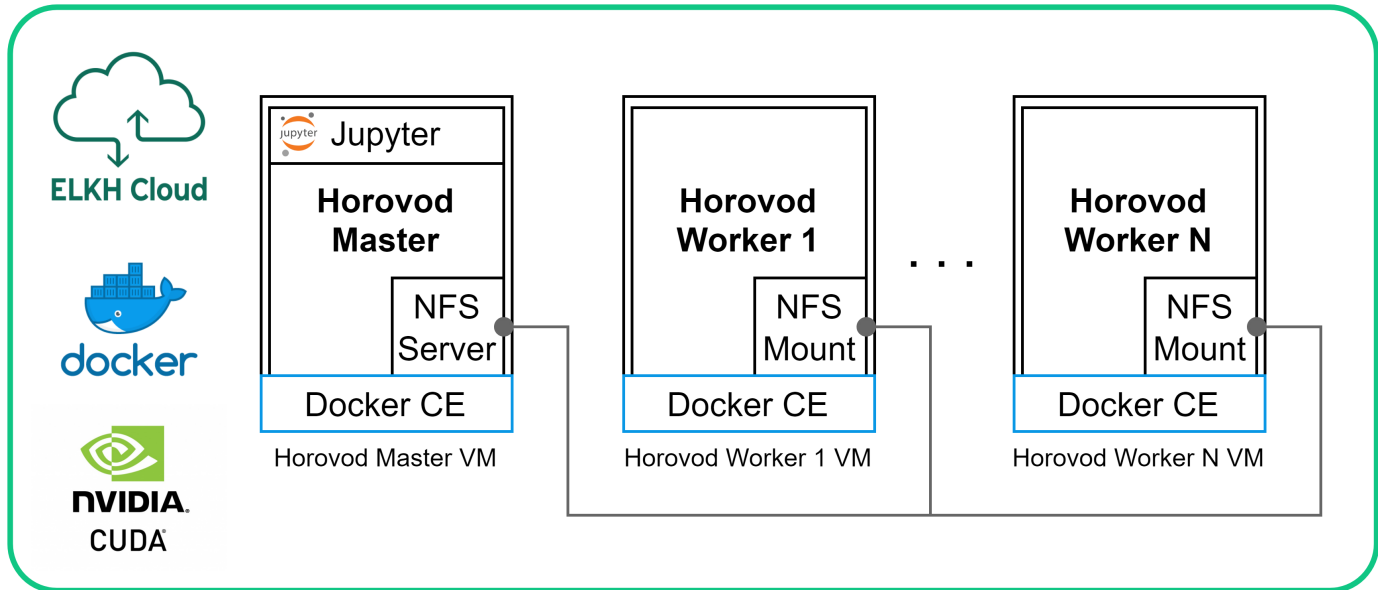


Fig. 1. Horovod reference architecture

the scalability of the cluster, we used 1 to 8 virtual machines at the same time. We used batch sizes of 32 and 196 for the experiments in order to observe how the batch size will affect the scalability and the accuracy of the training. The 32 size is the recommended default size and the 196 size is the maximum which can fit into the memory of one of our GPU cards. For the training we used the CIFAR-10 [25] and ImageNet [26] datasets as a smaller and larger input training data. The sizes of the CIFAR-10 and ImageNet datasets we worked with were 178MB and 141GB respectively. In the course of our investigation, we measured the processing speed of the network in images/second, the scaling efficiency and the training accuracy of the network.

IV. RESULTS

During the experiments, we performed each measurement five times and then presented the median of the results.

A. Scaling

First of all, we evaluated the scalability of the Horovod cluster from 1 to 8 nodes with different batch sizes and datasets. To measure the scaling, we processed 250 batches of data with the model and recorded the observed processing speed of the cluster. In this experiment, the accuracy of the model was not evaluated. The results of the ImageNet-32 and ImageNet-196 cases where we used the ImageNet dataset with 32 or 196 batch size are presented on the Figure 2 and Figure 3.

The processing speed of a single node was taken as the baseline value for these measurements. Ideal values represent the theoretical results of scaling with 100% efficiency – which would mean that distributing the training over N nodes results in N times the performance of a single one. The ideal values

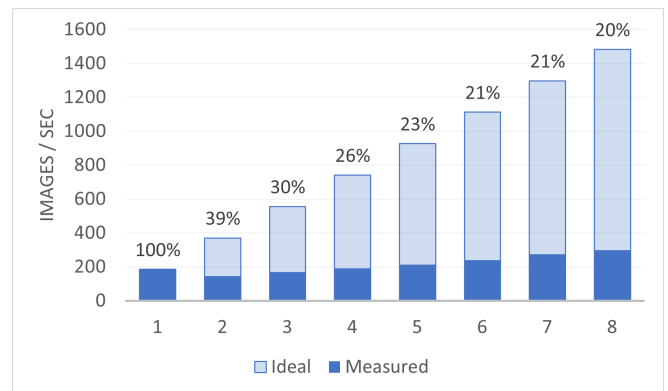


Fig. 2. Scalability results (including efficiency) with ImageNet dataset and 32 batch size running on 1 to 8 nodes

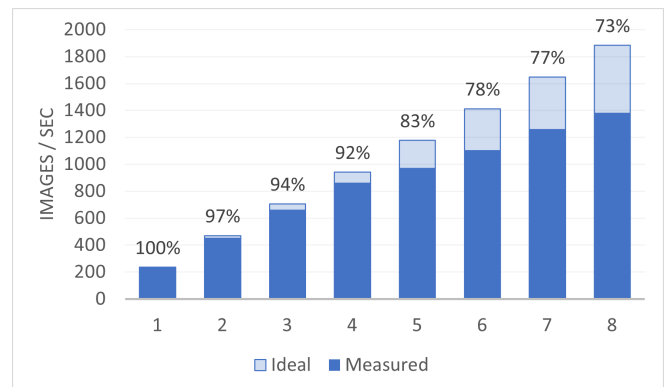


Fig. 3. Scalability results (including efficiency) with ImageNet dataset and 196 batch size running on 1 to 8 nodes

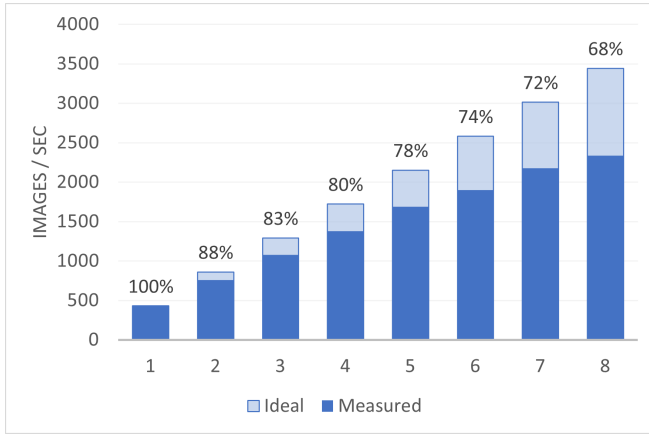


Fig. 4. Scalability results (including efficiency) with CIFAR-10 dataset and 32 batch size running on 1 to 8 nodes

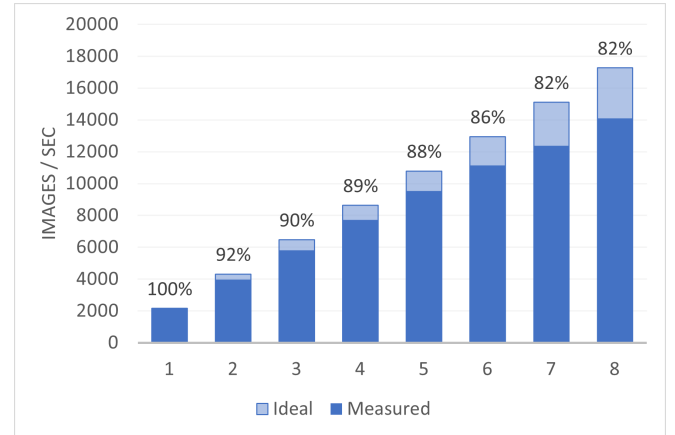


Fig. 5. Scalability results (including efficiency) with CIFAR-10 dataset and 196 batch size running on 1 to 8 nodes

are, and should be taken as theoretical, as in a realistic environment many problematic factors, such as communication overhead are present. As it can be observed, the batch size has a significant effect on the scaling efficiency. With a batch size of 32, the processing speed in the 2 and 3 node cases is even slower than in the 1 node case. With 8 nodes, the processing speed is just 1.6 times faster than in the 1 node case, which is only a 20% scaling efficiency. On the other hand, with the maximum 196 batch size, scaling efficiency improves considerably. With this higher batch size, the 2 node and 3 node scaling efficiency is nearly linear. As presented on the figure, the scaling efficiency using 8 nodes is 73%, which is 3.65 times better than in the 32 batch size case. These results show that batch size, which takes effect on the communication and computation time of each step, affects the scaling efficiency dramatically. We measured the scaling efficiency with CIFAR-10 as well, which is a significantly smaller dataset, with the same number of batches and configuration. These measurements are not realistic examples due to the size of the dataset and the kind and amount of computational resources used, but we wanted to examine the effect of the size of the dataset on the scaling efficiency. The results of the CIFAR-10-32 and CIFAR-10-196 cases are presented on the Figure 4 and 5.

The results are compiled in the same format. As it is presented, the scaling efficiency with the 32 batch size is better than that of the ImageNet dataset with the same batch size. This is due to the significantly smaller size of the dataset, as in this case the communication overhead is also smaller, therefore the scaling efficiency is better. The higher 196 batch size showed improved efficiency as well. The same measurements were also conducted with the CIFAR-10 dataset using a batch size of 512, the results of which followed a similar trend. This experiment showed that a larger batch size can considerably improve the scaling efficiency. With this dataset size and batch size however, the model can be easily over-fitted. These results also prove that the rate of communication and the computation time are key parameters in the matter of the scaling efficiency.

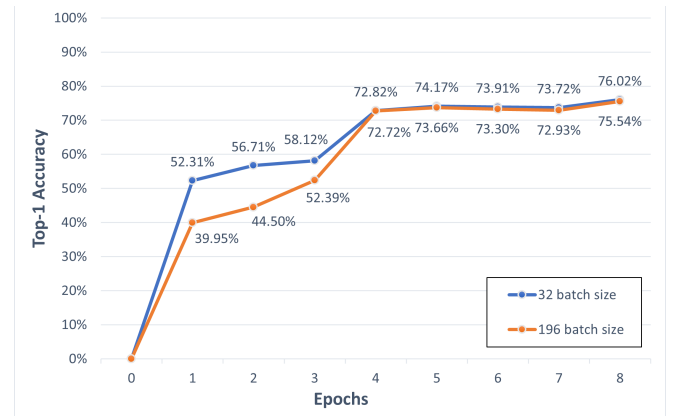


Fig. 6. Number of epoch of training a ResNet101 model on ImageNet to up to 75% top-1 accuracy

B. Accuracy

We aimed to measure how distributing the training and parameters such as batch size affect the accuracy of the trained model. For this experiment, we used the same cluster configuration with 8 nodes, the ImageNet dataset with the 32 and 196 batch size and with the target accuracy set to 75%. In this case we did not measure the processing speed, but instead evaluated the accuracy of the trained model after each epoch. As presented on the Figure 6, the accuracy of the model reached the target 75% after 8 epochs in both cases, which means that the distribution of the training did not affect the accuracy.

The accuracy after the first epoch was significantly better in the 32 batch size case, but following the fourth epoch the accuracy was nearly equal in both cases, up to the end of the training. We measured the runtime of the training in both cases, and the results are shown on Figure 7.

Based on these measurements, it can be concluded that the batch size has a significant effect on the training time as well. As it can be observed, the training time of the 32 batch case was 4.2 times slower than the 196 batch size, with the same

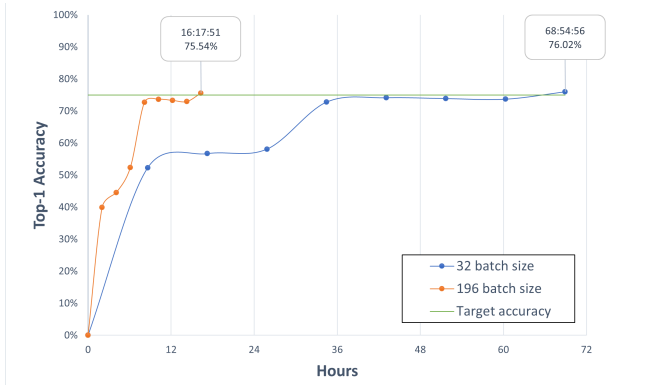


Fig. 7. Training time of a ResNet101 model on ImageNet to up to 75% top-1 accuracy

target accuracy. The actual runtime of the training can differ between different training processes because it depends on the initial state of the model as well, but we measured nearly the same difference between the two batch sizes after each experiment. These results also show the effect of the batch size on scalability.

C. Evaluation

Based on the presented results, a DDL framework can be effectively used and scaled from 1 to 8 nodes on general purpose infrastructure such as the ELKH Cloud with appropriate parameters. As the scaling efficiency measurements showed, the batch size and the size of the dataset have a significant effect on the scalability and the training time as well. One of the key aspects is the rate of computation and communication time of one step. With a smaller batch size, the computation time will be shorter, and communication will occur more frequently, which due to the overhead has a negative effect on the scalability. On the other hand, with a larger batch size, the scalability will improve but the classification accuracy of the model may decrease. This trade-off is presented in the accuracy measurements in the first three epochs. These results also confirm our observation that training accuracy is not affected by distributing the training process.

V. CONCLUSION AND FUTURE WORK

In this paper, we evaluated the Horovod DDL framework on the ELKH Cloud research infrastructure. We briefly introduced the DDL methodology and the used Horovod framework. We described the ELKH Cloud environment and the Horovod based reference architecture which was the base framework for the measurements. We utilized up to 8 nodes and used different input datasets and batch sizes for measurements of the scaling efficiency. We presented that the batch size has a significant effect on the rate of the computational and communication time between the nodes, which influences the scaling capability of the whole cluster. Furthermore, we validated that distributing the training does not have a negative effect on the training accuracy of the model. These results

present fairly that with the right parameters, a general purpose infrastructure can be used for DDL applications.

On behalf of the European Commissions Horizon 2020 project NEANIAS, encouraged by the promising results presented, the Horovod reference architecture has been made available as a service on the European Open Science Cloud (EOSC) Marketplace under the name 'Distributed Deep Learning by Horovod'.

Taking the presented results into consideration, our plans for the future regarding this area include the modelling of the DDL process, and exploring and experimenting with new possibilities within the training of deep neural networks. Our main goal is improving the scalability and efficiency of the process in given use-cases.

VI. ACKNOWLEDGMENT

The research leading to these results has received funding from the European Commissions Horizon 2020 research and innovation programme under the grant agreement No. 863448 (NEANIAS). This work was partially funded by the Hungarian Ministry of Innovation and Technology NRDI Office within the framework of the Artificial Intelligence National Laboratory Program. The work was supported in part by the Eötvös Loránd Research Network Secretariat (Development of cyber-medical systems based on AI and hybrid cloud methods) under Agreement ELKH KÖ-37/2021. On behalf of Project "Distributed deep learning in parallel and distributed environment" we thank for the usage of ELKH Cloud (<https://science-cloud.hu/>) that significantly helped us achieving the results published in this paper. The authors thankfully acknowledge the support of the Doctoral School of Applied Informatics and Applied Mathematics, Óbuda University. The presented work was supported by the UNKP-21-5 New National Excellence Program of the Ministry For Innovation And Technology from the source of the National, Research, Development and Innovation Fund.

REFERENCES

- [1] M. Samovský, V. Maslej-Krešňáková, and K. Ivancová, "Fake news detection related to the covid-19 in slovak language using deep learning methods," *Acta Polytechnica Hungarica*, vol. 19, no. 2, 2022.
- [2] R. Sedona, G. Cavallaro, J. Jitsev, A. Strube, M. Riedel, and J. A. Benediktsson, "Remote sensing big data classification with high performance distributed deep learning," *Remote Sensing*, vol. 11, no. 24, 2019. [Online]. Available: <https://www.mdpi.com/2072-4292/11/24/3056>
- [3] T. Ben-Nun and T. Hoeffer, "Demystifying parallel and distributed deep learning: An in-depth concurrency analysis," *ACM Computing Surveys (CSUR)*, vol. 52, no. 4, pp. 1–43, 2019.
- [4] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org. [Online]. Available: <https://www.tensorflow.org/>

- [5] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems* 32, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [6] Facebook, "Gloo," Available at <https://github.com/facebookincubator/gloo> (2022/04/25).
- [7] M. P. Forum, "Mpi: A message-passing interface standard," USA, Tech. Rep., 1994.
- [8] NVIDIA, "Nvidia nccl," Available at <https://developer.nvidia.com/nccl> (2022/04/25).
- [9] A. Sergeev and M. Del Balso, "Horovod: fast and easy distributed deep learning in tensorflow," 2018. [Online]. Available: <https://arxiv.org/abs/1802.05799>
- [10] R. Sedona, G. Cavallaro, M. Riedel, and M. Book, "Enhancing large batch size training of deep models for remote sensing applications," in *2021 IEEE International Geoscience and Remote Sensing Symposium IGARSS*, 2021, pp. 1583–1586.
- [11] F. Chollet *et al.*, "Keras," <https://keras.io>, 2015.
- [12] T. Chen, M. Li, Y. Li, M. Lin, N. Wang, M. Wang, T. Xiao, B. Xu, C. Zhang, and Z. Zhang, "Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems," 2015. [Online]. Available: <https://arxiv.org/abs/1512.01274>
- [13] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, p. 56–65, oct 2016. [Online]. Available: <https://doi.org/10.1145/2934664>
- [14] Baidu, "Bringing hpc techniques to deep learning," Available at <https://andrew.gibiansky.com/blog/machine-learning/baidu-allreduce/> (2022/04/25).
- [15] NVIDIA, "Nvidia tesla v100 - the first tensor core gpu," Available at <https://www.nvidia.com/en-gb/data-center/tesla-v100/> (2022/04/25).
- [16] OpenStack, "The most widely deployed open source cloud software in the world," Available at <https://www.openstack.org/> (2022/04/25).
- [17] S. A. Weil, S. A. Brandt, E. L. Miller, D. D. E. Long, and C. Maltzahn, "Ceph: A scalable, high-performance distributed file system," in *Proceedings of the 7th Symposium on Operating Systems Design and Implementation*, ser. OSDI '06. USA: USENIX Association, 2006, p. 307–320.
- [18] "Horovod reference architecture," Available at <https://science-cloud.hu/en/reference-architectures/horovod-cluster> (2022/04/25).
- [19] GitLab, "Gitlab continuous integration (ci)," Available at <https://about.gitlab.com/stages-devops-lifecycle/continuous-integration/> (2022/04/25).
- [20] Terraform, "Deliver infrastructure as code," Available at <https://www.terraform.io/> (2022/04/25).
- [21] Ansible, "Ansible documentation," Available at <https://docs.ansible.com/ansible/latest/index.html> (2022/04/25).
- [22] Jupyter, "Jupyterlab documentation," Available at <https://jupyterlab.readthedocs.io/en/stable/> (2022/04/25).
- [23] S. Microsystems, "Rfc1094: Nfs: Network file system protocol specification," USA, 1989.
- [24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015. [Online]. Available: <http://arxiv.org/abs/1512.03385>
- [25] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., 2009.
- [26] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision (IJCV)*, vol. 115, no. 3, pp. 211–252, 2015.